

## Performance Testing

|                      |                          |
|----------------------|--------------------------|
| <b>Date</b>          | 28 September 2026        |
| <b>Team ID</b>       | 6ab4d2876603573ccfb56348 |
| <b>Project Name</b>  | AI WEATHERWISE           |
| <b>Maximum Marks</b> | 5 Marks                  |

### Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

#### Step 1: Testing Overview

| Field                      | Details                                                                                            |
|----------------------------|----------------------------------------------------------------------------------------------------|
| <b>Testing Tool Used</b>   | Postman / Apache JMeter                                                                            |
| <b>Type of Testing</b>     | Load Testing & API Performance Evaluation                                                          |
| <b>Target Module / API</b> | Weather API Endpoints (/api/weather), Auth Endpoints (/api/auth), and AI Recommendations (/api/ai) |
| <b>Test Environment</b>    | Local Node.js Server (http://localhost:5500) connected to MongoDB Cloud Database                   |
| <b>Test Date</b>           | 28 September 2026                                                                                  |

#### Step 2: Test Scenarios

| S.No | Test Scenario / Description                                                                              | No. of Virtual Users | Duration (sec) | Expected Outcome                                                    |
|------|----------------------------------------------------------------------------------------------------------|----------------------|----------------|---------------------------------------------------------------------|
| 1    | <b>User Login &amp; JWT Token Generation</b><br><br>Simulate concurrent user logins via /api/auth/login. | 50                   | 60             | All requests succeed with valid JWT tokens returned under 1 second. |

|   |                                                                                                 |     |     |                                                                                        |
|---|-------------------------------------------------------------------------------------------------|-----|-----|----------------------------------------------------------------------------------------|
| 2 | <b>Real-Time Weather Query</b><br>Concurrent calls to /api/weather fetching live weather data.  | 100 | 120 | Successful response with weather data structure and average latency \$< 1.5\$ seconds. |
| 3 | <b>I Weather Recommendations</b><br>Simulate requests to /api/ai/recommendations.               | 20  | 60  | AI responses returned without backend server timeouts or unhandled exceptions          |
| 4 | <b>Saved Locations Persistence</b><br>Simulate saving and reading locations via /api/locations. | 50  | 60  | Database CRUD operations complete successfully with error rate \$< 1\%\$.              |

### Step 3: Performance Test Results

| S.No | Metric               | Target Value | Actual Value | Status (Pass / Fail) | Remarks                                                           |
|------|----------------------|--------------|--------------|----------------------|-------------------------------------------------------------------|
| 1    | Response Time (Avg)  | < 2 seconds  | 450 ms       | Pass                 | API endpoints responded quickly under normal loads.               |
| 2    | Response Time (Max)  | < 5 seconds  | 1.85 seconds | Pass                 | External API call latency remained well within boundaries.        |
| 3    | Throughput (Req/sec) | > 25 req/sec | 42 req/sec   | Pass                 | Express server efficiently handled concurrent incoming API calls. |
| 4    | Error Rate           | < 1%         | 0.00%        | Pass                 | Zero unhandled crashes or HTTP 500 errors observed.               |
| 5    | CPU Utilization      | < 80%        | 32%          | Pass                 | Node.js process CPU usage remained optimal.                       |
| 6    | Memory Utilization   | < 80%        | 45%          | Pass                 | Memory consumption was stable without memory leaks.               |

### Step 4: Observations & Analysis

- **API Latency & Stability:** The Node.js and Express backend maintained fast response times for authentication and database queries, operating well within the maximum threshold of 5 seconds.
- **Third-Party Service Integration:** Requests involving external API integration (Weather API and AI Service) contributed slightly higher response times than local database queries, but remained well within acceptable performance bounds.

- **Database Performance:** MongoDB connection pool handling in `db.js` proved reliable under load, preventing bottlenecks during concurrent location retrieval and save actions.
- **Error Handling Integrity:** Input validation middleware prevented invalid requests from impacting server performance or producing unhandled crashes.

### Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

